THE UNIVERSITY OF AZAD JAMMU AND KASHMIR, MUZAFFARABAD

LAB 02

FROM REAL WORLD CONCEPT TO SOFTWARE OBJECT (COMPANY MANAGEMENT SYSTEM)

SUBMISSION DETAILS	
Student Name	Khawaja Absar Ali
Roll Number	2025-SE-31
Course Code	SEC-532
Course Name	Software Design & Architecture
Instructor	Engr. Ghulam Rabbani Butt
Semester	3rd
Session	2025–29
Lab Title	From Real World Concept to Software Object (Company Management System)
Assigned On	October 21, 2026
Due Date	October 28, 2026
Submission Date	October 26, 2026
University	The University of Azad Jammu and Kashmir

Company Management System: Conceptual Classes and Abstraction

1. Introduction and Selected Domain

For this assignment I selected the Company Management System as my real world domain. A company has to look after many things at the same time, such as employees, departments, clients, projects, tasks, expenses, invoices, payments, attendance and leave requests. When these are handled without a proper structure, work gets delayed and information gets lost. The purpose of this system is to organize these activities in one place so that daily company operations become easier to manage.

2. Candidate Conceptual Classes

I identified the following 10 conceptual classes. They are conceptual classes because each one stands for a meaningful entity or idea that exists in the company management domain, and not for a screen, a table or a piece of code.

1. **Employee**: A person working for the company. It stores employment details and helps track responsibilities and assigned tasks.
2. **Department**: A business unit such as Development, Design or Marketing. It organizes employees according to their roles and teams.
3. **Client**: A person or business that hires the company for services. It stores contact information and helps maintain client relationships.
4. **Project**: A piece of work completed for a client. It tracks the project name, deadline, status and related tasks.
5. **Task**: A specific activity within a project. It identifies the work to be done, the responsible employee and the task status.
6. **Expense**: Money spent by the company. It records the amount, date, category and purpose of each expense.
7. **Invoice**: A bill sent to a client. It records the amount charged, issue date, due date and payment status.
8. **Payment**: Money received from a client or paid to a supplier. It records the amount, date and payment method.
9. **AttendanceRecord**: An employee's attendance on a particular working day. It stores the attendance status and relevant time information.
10. **LeaveRequest**: An employee's request for time off. It stores the requested dates, reason and approval status.

3. Attributes and Methods of Three Important Classes

Attributes are the pieces of information a class stores, while methods are the actions or behavior the class can perform. I chose Employee, Project and Expense for detailed modeling.

3.1 Employee Class

Attributes

- **employeeId**: A unique identifier used to distinguish one employee from another.
- **fullName**: Stores the employee's complete name.
- **workEmail**: Stores the employee's official company email address.
- **department**: Identifies the department to which the employee belongs.
- **jobTitle**: Stores the employee's position in the company.

Methods

- **updateProfile()**: Updates the employee's personal or professional details.
- **viewAssignedTasks()**: Displays the tasks currently assigned to the employee.
- **submitLeaveRequest()**: Allows the employee to submit a request for leave.
- **viewAttendance()**: Displays the employee's attendance records.

3.2 Project Class

Attributes

- **projectId**: A unique identifier for each project.
- **projectName**: Stores the name of the project.
- **clientId**: Identifies the client for whom the project is being completed.
- **startDate**: Records the date when the project begins.
- **deadline**: Stores the expected completion date.
- **status**: Shows the current progress, such as planned, in progress or completed.

Methods

- **createProject()**: Creates a new project record.
- **updateStatus()**: Changes the project's current status.
- **assignTask()**: Assigns a task to an appropriate team member.
- **calculateProgress()**: Estimates project completion using the status of its tasks.

3.3 Expense Class

Attributes

- **expenseId**: A unique identifier for each expense record.
- **description**: Explains the reason for the expense.
- **amount**: Stores the amount of money spent.
- **expenseDate**: Records the date on which the expense occurred.
- **category**: Identifies the expense type, such as software, equipment or travel.

Methods

- **recordExpense()**: Saves a new expense in the system.
- **updateExpense()**: Updates the details of an existing expense.
- **submitForApproval()**: Sends the expense for review by an authorized person.
- **generateExpenseSummary()**: Produces a summary of recorded expenses for financial review.

4. Main Abstraction Decisions

Employee, Project and Expense were selected for detailed modeling because they are central to the daily operations of a company. The other classes act as supporting concepts that help manage teams, clients, work activities and payments. Payroll calculations, tax filing, recruitment, inventory management and detailed accounting were deliberately excluded because they are beyond the scope of this small model. Screens, database tables, login controllers and programming language details were also left out, since conceptual modeling focuses on the real world business domain and not on technical implementation.

5. Invented Software Object

Expense Approval Validator

An Expense Approval Validator is an invented software object. It does not represent a physical entity in the company, but it is still useful in the design.

- **Purpose:** Checks an expense submission before it is approved.
- **Validation:** Checks whether the expense amount is valid, the description and category are provided, and a required receipt is attached.
- **Result:** Returns whether the expense can proceed or identifies the missing information.
- **Benefit:** Keeps validation rules consistent and reduces incomplete or incorrect expense submissions.

Even though it has no direct counterpart in the real world, this object shows that software design sometimes needs objects created only to coordinate processes or enforce business rules.

6. Conclusion

The Company Management System is a practical example of conceptual modeling. By identifying classes, defining their attributes and methods, and making clear abstraction decisions, we can organize a software system around real business needs before thinking about implementation.